

**This task aim is to Build and evaluate a supervised classification model on spam detection**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report
)
```

## 1 LOADING THE DATA

```
df = pd.read_csv("spam.csv")

df.head()
```

|   | Category | Message                                           |  |
|---|----------|---------------------------------------------------|--|
| 0 | ham      | Go until jurong point, crazy.. Available only ... |  |
| 1 | ham      | Ok lar... Joking wif u oni...                     |  |
| 2 | spam     | Free entry in 2 a wkly comp to win FA Cup fina... |  |
| 3 | ham      | U dun say so early hor... U c already then say... |  |
| 4 | ham      | Nah I don't think he goes to usf, he lives aro... |  |

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
print(df.shape)

df.info()

df.isnull().sum()
```

```
(5572, 2)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Category    5572 non-null   object
1   Message     5572 non-null   object
dtypes: object(2)
memory usage: 87.2+ KB
```

```

      0
Category 0
Message 0

dtype: int64
```

```
X = df['Message']
y = df['Category']
```

```
vectorizer = TfidfVectorizer(stop_words='english')

X = vectorizer.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
lr = LogisticRegression()

lr.fit(X_train, y_train)

lr_pred = lr.predict(X_test)
```

```
print("Accuracy:",
      accuracy_score(y_test, lr_pred))

print("Precision:",
      precision_score(y_test, lr_pred, pos_label='spam'))

print("Recall:",
      recall_score(y_test, lr_pred, pos_label='spam'))

print("F1:",
      f1_score(y_test, lr_pred, pos_label='spam'))
```

```
Accuracy: 0.95695067264574
Precision: 1.0
Recall: 0.6778523489932886
F1: 0.808
```

```
rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)
```

```

print("Accuracy:",
      accuracy_score(y_test, rf_pred))

print("Precision:",
      precision_score(y_test, rf_pred, pos_label='spam'))

print("Recall:",
      recall_score(y_test, rf_pred, pos_label='spam'))

print("F1:",
      f1_score(y_test, rf_pred, pos_label='spam'))

```

```

Accuracy: 0.9811659192825112
Precision: 1.0
Recall: 0.8590604026845637
F1: 0.924187725631769

```

```

lr_prob = lr.predict_proba(X_test)[: ,1]

rf_prob = rf.predict_proba(X_test)[: ,1]

print("LR ROC-AUC:",
      roc_auc_score(y_test, lr_prob))

print("RF ROC-AUC:",
      roc_auc_score(y_test, rf_prob))

```

```

LR ROC-AUC: 0.9883905123181458
RF ROC-AUC: 0.9928682590631817

```

```

cv_lr = cross_val_score(
    lr,
    X,
    y,
    cv=5
)

print("LR CV Accuracy:",
      cv_lr.mean())

cv_rf = cross_val_score(
    rf,
    X,
    y,
    cv=5
)

print("RF CV Accuracy:",
      cv_rf.mean())

```

```

LR CV Accuracy: 0.9529787216913155
RF CV Accuracy: 0.9759491510413731

```

```

lr_acc = accuracy_score(y_test, lr_pred)
rf_acc = accuracy_score(y_test, rf_pred)

models = [
    "Logistic Regression",
    "Random Forest"
]

scores = [
    lr_acc,
    rf_acc
]

```

```
plt.figure(figsize=(6, 4))
```

```
plt.figure(figsize=(8,4))  
plt.bar(models, scores)  
  
plt.title("Model Accuracy Comparison")  
plt.ylabel("Accuracy")  
  
plt.show()
```

